IPFS 커스텀 플러그인 개발 가이드

개요

이 가이드는 IPFS Kubo용 커스텀 플러그인을 개발하는 방법을 단계별로 설명합니다. 플러그인 개발부터 테스트, 배포까지의 전체 과정을 다루며, 각 플러그인 타입별 구체적인 구현 방법과 모범 사례를 제공합니다.

플러그인 개발 기본 구조

1. 기본 프로젝트 구조

```
my-custom-plugin/
├── go.mod # Go 모듈 정의
├── go.sum # 의존성 체크섬
├── plugin.go # 메인 플러그인 구현
├── README.md # 플러그인 문서
├── examples/ # 사용 예제
| └── config.json # 설정 예제
└── tests/ # 테스트 파일들
├── plugin_test.go # 단위 테스트
└── integration_test.go # 통합 테스트
```

2. 기본 플러그인 인터페이스 구현

모든 플러그인은 기본 `Plugin` 인터페이스를 구현해야 합니다:

```
package main

import (
"github.com/ipfs/kubo/plugin"
)

// Plugins는 로더가 찾을 수 있도록 내보내진 플러그인 목록입니다
var Plugins = []plugin.Plugin{
&myCustomPlugin{},
}

type myCustomPlugin struct {
// 플러그인 상태 필드들
config map[string]interface{}
repo string
}

// 기본 Plugin 인터페이스 구현
func (p *myCustomPlugin) Name() string {
return "my-custom-plugin"
}
```

```
func (p *myCustomPlugin) Version() string {
return "1.0.0"
}

func (p *myCustomPlugin) Init(env *plugin.Environment) error {
p.repo = env.Repo
p.config = make(map[string]interface{})
// 설정이 제공된 경우 파싱
if env.Config != nil {
if configMap, ok := env.Config.(map[string]interface{}); ok {
p.config = configMap
}
}
return nil
}
```

플러그인 타입별 개발 가이드

1. 데이터스토어 플러그인 개발

데이터스토어 플러그인은 새로운 저장소 백엔드를 추가할 때 사용됩니다.

기본 구조

```go
package main


import (
"fmt"
"path/filepath"
"github.com/ipfs/kubo/plugin"
"github.com/ipfs/kubo/repo"
"github.com/ipfs/kubo/repo/fsrepo"
)


type customDatastorePlugin struct{}


var _ plugin.PluginDatastore = (*customDatastorePlugin)(nil)


func (p *customDatastorePlugin) Name() string {
return "custom-datastore"
}


func (p *customDatastorePlugin) Version() string {
return "1.0.0"
}
```

```
func (p *customDatastorePlugin) Init(env *plugin.Environment) error {
return nil
}

func (p *customDatastorePlugin) DatastoreTypeName() string {
return "customds"
}

func (p *customDatastorePlugin) DatastoreConfigParser() fsrepo.ConfigFromMap {
return func(params map[string]interface{}) (fsrepo.DatastoreConfig, error) {
return parseDatastoreConfig(params)
}
}
```

설정 파서 구현

```
type datastoreConfig struct {
path string
cacheSize int64
compression bool
}

func parseDatastoreConfig(params map[string]interface{}) (fsrepo.DatastoreConf
var c datastoreConfig
```

```go
// 필수 파라미터: path
path, ok := params["path"].(string)
if !ok {
return  nil, fmt.Errorf("'path' field is missing or not string")
}
c.path = path
// 선택적 파라미터: cacheSize
if  cacheSize, ok := params["cacheSize"].(float64);  ok {
c.cacheSize = int64(cacheSize)
} else {
c.cacheSize = 1024 * 1024 * 100  // 기본값: 100MB
}
// 선택적 파라미터: compression
if  compression, ok := params["compression"].(bool);  ok {
c.compression = compression
} else {
c.compression = true // 기본값: 압축 활성화
}
return &c, nil
}


// fsrepo.DatastoreConfig 인터페이스 구현
func (c *datastoreConfig) DiskSpec() fsrepo.DiskSpec {
```

```
return  map[string]interface{}{
"type": "customds",
"path": c.path,
"cacheSize": c.cacheSize,
"compression": c.compression,
}
}

func (c *datastoreConfig) Create(path  string) (repo.Datastore, error) {
p := c.path
if !filepath.IsAbs(p) {
p = filepath.Join(path, p)
}
// 실제 데이터스토어 인스턴스 생성
return  NewCustomDatastore(p, c.cacheSize, c.compression)
}
```

실제 데이터스토어 구현

```
import (
"github.com/ipfs/go-datastore"
"github.com/ipfs/go-datastore/query"
)
```

```go
type CustomDatastore struct {
path  string
cacheSize  int64
compression  bool
// 내부 구현 필드들...
}

func  NewCustomDatastore(path  string, cacheSize  int64, compression  bool) (*Cu
// 데이터스토어 초기화 로직
return &CustomDatastore{
path: path,
cacheSize: cacheSize,
compression: compression,
}, nil
}

// datastore.Datastore 인터페이스 구현
func (ds *CustomDatastore) Get(ctx context.Context, key datastore.Key) ([]byte,
// 데이터 조회 구현
}

func (ds *CustomDatastore) Put(ctx context.Context, key datastore.Key, value []b
```

```
// 데이터 저장 구현
}

func (ds *CustomDatastore) Delete(ctx context.Context, key datastore.Key) error
// 데이터 삭제 구현
}

func (ds *CustomDatastore) Query(ctx context.Context, q query.Query) (query.Resu
// 쿼리 구현
}

func (ds *CustomDatastore) Close() error {
// 리소스 정리
}
```

2. IPLD 코덱 플러그인 개발

IPLD 플러그인은 새로운 데이터 형식과 코덱을 추가할 때 사용됩니다.

기본 구조

```
package main

import (
```

```go
"io"

"github.com/ipfs/kubo/plugin"

"github.com/ipld/go-ipld-prime"

"github.com/ipld/go-ipld-prime/multicodec"

mc  "github.com/multiformats/go-multicodec"

)


type customIPLDPlugin struct{}


var  _ plugin.PluginIPLD = (*customIPLDPlugin)(nil)


func (p *customIPLDPlugin) Name() string {

return  "custom-ipld-codec"

}


func (p *customIPLDPlugin) Version() string {

return  "1.0.0"

}


func (p *customIPLDPlugin) Init(env *plugin.Environment) error {

return  nil

}
```

```go
func (p *customIPLDPlugin) Register(reg multicodec.Registry) error {
// 커스텀 코덱 등록
reg.RegisterEncoder(uint64(mc.ReservedStart+100), encodeCustomFormat)
reg.RegisterDecoder(uint64(mc.ReservedStart+100), decodeCustomFormat)
return nil
}
```

인코더/디코더 구현

```go
func encodeCustomFormat(node ipld.Node, w io.Writer) error {
// 커스텀 형식으로 노드 인코딩
data, err := serializeNode(node)
if err != nil {
return err
}
_, err = w.Write(data)
return err
}

func decodeCustomFormat(na ipld.NodeAssembler, r io.Reader) error {
// 커스텀 형식에서 노드 디코딩
data, err := io.ReadAll(r)
if err != nil {
return err
```

```
}
return deserializeToNode(data, na)
}

func serializeNode(node ipld.Node) ([]byte, error) {
// 노드를 커스텀 바이너리 형식으로 직렬화
// 실제 구현은 사용하는 데이터 형식에 따라 달라집니다
}

func deserializeToNode(data []byte, na ipld.NodeAssembler) error {
// 바이너리 데이터를 노드로 역직렬화
// 실제 구현은 사용하는 데이터 형식에 따라 달라집니다
}
```

3. 데몬 플러그인 개발

데몬 플러그인은 IPFS 노드와 함께 실행되는 서비스를 구현할 때 사용됩니다.

외부 API 접근 플러그인 (PluginDaemon)

```
package main

import (
"context"
```

```
"time"

"github.com/ipfs/kubo/plugin"

"github.com/ipfs/kubo/core/coreiface"

logging  "github.com/ipfs/go-log/v2"

)


var  log = logging.Logger("custom-daemon-plugin")


type customDaemonPlugin struct {

api coreiface.CoreAPI

ctx context.Context

cancel context.CancelFunc

config  map[string]interface{}

}


var  _ plugin.PluginDaemon = (*customDaemonPlugin)(nil)


func (p *customDaemonPlugin) Name() string {

return  "custom-daemon-service"

}


func (p *customDaemonPlugin) Version() string {

return  "1.0.0"
```

```
}


func (p *customDaemonPlugin) Init(env *plugin.Environment) error {
if  env.Config != nil {
if  configMap, ok := env.Config.(map[string]interface{});  ok {
p.config = configMap
}
}
return  nil
}


func (p *customDaemonPlugin) Start(api coreiface.CoreAPI) error {
p.api = api
p.ctx, p.cancel = context.WithCancel(context.Background())
// 백그라운드 서비스 시작
go  p.runService()
log.Info("Custom daemon service started")
return  nil
}


func (p *customDaemonPlugin) Close() error {
if  p.cancel != nil {
p.cancel()
```

```
}
log.Info("Custom daemon service stopped")
return nil
}


func (p *customDaemonPlugin) runService() {
ticker := time.NewTicker(30 * time.Second)
defer ticker.Stop()
for {
select {
case <-p.ctx.Done():
return
case <-ticker.C:
// 주기적인 작업 수행
p.performPeriodicTask()
}
}
}


func (p *customDaemonPlugin) performPeriodicTask() {
// CoreAPI를 사용한 IPFS 작업
peers, err := p.api.Swarm().Peers(p.ctx)
if err != nil {
```

```
log.Errorf("Failed to get peers: %v", err)

return

}

log.Infof("Connected to %d peers", len(peers))

// 추가 로직...

}
```

내부 API 접근 플러그인 (PluginDaemonInternal)

```
package main


import (

"context"

"github.com/ipfs/kubo/plugin"

"github.com/ipfs/kubo/core"

"github.com/libp2p/go-libp2p/core/event"

"github.com/libp2p/go-libp2p/core/network"

"github.com/libp2p/go-libp2p/core/peer"

)


type customInternalPlugin struct {

node *core.IpfsNode

ctx context.Context

cancel context.CancelFunc
```

```
}

var _ plugin.PluginDaemonInternal = (*customInternalPlugin)(nil)

func (p *customInternalPlugin) Name() string {
return "custom-internal-service"
}

func (p *customInternalPlugin) Version() string {
return "1.0.0"
}

func (p *customInternalPlugin) Init(env *plugin.Environment) error {
return nil
}

func (p *customInternalPlugin) Start(node *core.IpfsNode) error {
p.node = node
p.ctx, p.cancel = context.WithCancel(node.Context())
// 이벤트 버스 구독
sub, err := node.PeerHost.EventBus().Subscribe(new(event.EvtPeerIdentificationCo
if err != nil {
return err
```

```
}
// 네트워크 이벤트 리스너 등록
var notifee network.NotifyBundle
notifee.ConnectedF = func(net network.Network, conn network.Conn) {
p.onPeerConnected(conn.RemotePeer())
}
notifee.DisconnectedF = func(net network.Network, conn network.Conn) {
p.onPeerDisconnected(conn.RemotePeer())
}
node.PeerHost.Network().Notify(&notifee)
// 이벤트 처리 고루틴 시작
go p.handleEvents(sub)
return nil
}

func (p *customInternalPlugin) Close() error {
if p.cancel != nil {
p.cancel()
}
return nil
}

func (p *customInternalPlugin) onPeerConnected(peerID peer.ID) {
```

```
log.Infof("Peer connected: %s", peerID.String())
// 연결 이벤트 처리 로직
}

func (p *customInternalPlugin) onPeerDisconnected(peerID peer.ID) {
log.Infof("Peer disconnected: %s", peerID.String())
// 연결 해제 이벤트 처리 로직
}

func (p *customInternalPlugin) handleEvents(sub event.Subscription) {
defer  sub.Close()
for {
select {
case <-p.ctx.Done():
return
case  e := <-sub.Out():
switch  evt := e.(type) {
case event.EvtPeerIdentificationCompleted:
p.onPeerIdentified(evt.Peer)
}
}
}
}
```

```
func (p *customInternalPlugin) onPeerIdentified(peerID peer.ID) {
// 피어 식별 완료 이벤트 처리
agent, err := p.node.Peerstore.Get(peerID, "AgentVersion")
if  err == nil {
if  agentStr, ok := agent.(string);  ok {
log.Infof("Peer %s identified with agent: %s", peerID.String(), agentStr)
}
}
}
```

4. FX 의존성 주입 플러그인 개발

FX 플러그인은 IPFS 노드의 의존성 그래프를 수정할 때 사용됩니다.

```
package main

import (
"github.com/ipfs/kubo/plugin"
"github.com/ipfs/kubo/core"
"go.uber.org/fx"
)

type customFxPlugin struct {
```

```
repo  string
}

var  _ plugin.PluginFx = (*customFxPlugin)(nil)

func (p *customFxPlugin) Name() string {
return  "custom-fx-plugin"
}

func (p *customFxPlugin) Version() string {
return  "1.0.0"
}

func (p *customFxPlugin) Init(env *plugin.Environment) error {
p.repo = env.Repo
return  nil
}

func (p *customFxPlugin) Options(info core.FXNodeInfo) ([]fx.Option, error) {
opts := info.FXOptions
// 커스텀 서비스 제공자 추가
opts = append(opts,
fx.Provide(NewCustomService),
```

```
fx.Invoke(StartCustomService),
)
// 기존 서비스 데코레이션
opts = append(opts,
fx.Decorate(WrapExistingService),
)
return  opts, nil
}


// 커스텀 서비스 정의
type CustomService struct {
config  map[string]interface{}
}


func  NewCustomService() *CustomService {
return &CustomService{
config: make(map[string]interface{}),
}
}


func  StartCustomService(service *CustomService) {
// 서비스 초기화 로직
log.Info("Custom service started via FX")
```

```
}


func WrapExistingService(original SomeExistingService) SomeExistingService {
// 기존 서비스를 래핑하여 기능 확장
return &WrappedService{
original: original,
}
}
```

플러그인 테스트

1. 단위 테스트

```
package main


import (
"testing"
"github.com/ipfs/kubo/plugin"
"github.com/stretchr/testify/assert"
"github.com/stretchr/testify/require"
)


func TestPluginBasics(t *testing.T) {
```

```
p := &myCustomPlugin{}
// 기본 인터페이스 테스트
assert.Equal(t, "my-custom-plugin", p.Name())
assert.Equal(t, "1.0.0", p.Version())
// 초기화 테스트
env := &plugin.Environment{
Repo: "/tmp/test-repo",
Config: map[string]interface{}{
"testOption": "testValue",
},
}
err := p.Init(env)
require.NoError(t, err)
assert.Equal(t, "/tmp/test-repo", p.repo)
}


func  TestDatastorePlugin(t *testing.T) {
p := &customDatastorePlugin{}
// 타입 이름 테스트
assert.Equal(t, "customds", p.DatastoreTypeName())
// 설정 파서 테스트
parser := p.DatastoreConfigParser()
config, err := parser(map[string]interface{}{
```

```
"path": "/test/path",
"cacheSize": float64(1024),
"compression": true,
})
require.NoError(t, err)
assert.NotNil(t, config)
}
```

2. 통합 테스트

```
package main

import (
"context"
"testing"
"time"
"github.com/ipfs/kubo/core"
"github.com/ipfs/kubo/plugin/loader"
"github.com/ipfs/kubo/repo/fsrepo"
"github.com/stretchr/testify/require"
)

func  TestPluginIntegration(t *testing.T) {
```

```
// 테스트용 IPFS 노드 생성
repo := createTestRepo(t)
defer  cleanupTestRepo(repo)
// 플러그인 로더 생성
pluginLoader, err := loader.NewPluginLoader(repo.GetPath())
require.NoError(t, err)
// 커스텀 플러그인 로드
err = pluginLoader.Load(&myCustomPlugin{})
require.NoError(t, err)
// 플러그인 초기화
err = pluginLoader.Initialize()
require.NoError(t, err)
// 플러그인 주입
err = pluginLoader.Inject()
require.NoError(t, err)
// IPFS 노드 생성
node, err := core.NewNode(context.Background(), &core.BuildCfg{
Repo: repo,
})
require.NoError(t, err)
defer  node.Close()
// 데몬 플러그인 시작
err = pluginLoader.Start(node)
```

```
require.NoError(t, err)

// 플러그인 기능 테스트

time.Sleep(100 * time.Millisecond) // 플러그인 시작 대기

// 여기서 플러그인의 실제 기능을 테스트

// 예: API 호출, 데이터 검증 등

// 정리

err = pluginLoader.Close()

require.NoError(t, err)

}


func  createTestRepo(t *testing.T) repo.Repo {

// 테스트용 임시 저장소 생성

// 실제 구현은 테스트 환경에 따라 달라집니다

}


func  cleanupTestRepo(repo repo.Repo) {

// 테스트 저장소 정리

}
```

3. 벤치마크 테스트

```
func  BenchmarkPluginPerformance(b *testing.B) {

p := &myCustomPlugin{}
```

```
env := &plugin.Environment{
Repo: "/tmp/bench-repo",
}
err := p.Init(env)
require.NoError(b, err)
b.ResetTimer()
for i := 0; i < b.N; i++ {
// 성능 측정할 작업 수행
performPluginOperation(p)
}
}

func performPluginOperation(p *myCustomPlugin) {
// 플러그인의 핵심 기능 실행
}
```

플러그인 빌드 및 배포

1. 정적 빌드 (프리로드)

플러그인을 IPFS 바이너리에 정적으로 포함시키는 방법:

1단계: 플러그인을 IPFS 소스에 추가

```
# IPFS 소스 디렉토리에서
```

```
mkdir -p plugin/plugins/mycustomplugin

cp -r /path/to/my-plugin/* plugin/plugins/mycustomplugin/
```

2단계: preload_list에 추가

```
echo "mycustomplugin github.com/ipfs/kubo/plugin/plugins/mycustomplugin *" >> p
```

3단계: 빌드

```
make build
```

2. 동적 빌드 (외부 플러그인)

플러그인을 별도의 공유 라이브러리로 빌드하는 방법:

go.mod 파일:

```
module github.com/myorg/my-ipfs-plugin


go 1.21


require (

github.com/ipfs/kubo v0.24.0

github.com/ipfs/go-datastore v0.6.0

// 기타 의존성들...
```

```
)
```

빌드 스크립트:

```
#!/bin/bash

# CGO 활성화 (동적 로딩에 필요)
export CGO_ENABLED=1

# 공유 라이브러리로 빌드
go build -buildmode=plugin -o my-custom-plugin.so

# 플러그인 디렉토리에 복사
cp my-custom-plugin.so ~/.ipfs/plugins/
```

Makefile 예시:

```
.PHONY: build install test clean

PLUGIN_NAME = my-custom-plugin
PLUGIN_SO = $(PLUGIN_NAME).so

build:
CGO_ENABLED=1 go build -buildmode=plugin -o $(PLUGIN_SO)
```

```
install: build
mkdir -p ~/.ipfs/plugins
cp $(PLUGIN_SO) ~/.ipfs/plugins/


test:
go test -v ./...


clean:
rm -f $(PLUGIN_SO)


# 크로스 컴파일 (Linux용)
build-linux:
GOOS=linux GOARCH=amd64 CGO_ENABLED=1 go build -buildmode=plugin -o $(PLUGIN_NAM


# 크로스 컴파일 (macOS용)
build-darwin:
GOOS=darwin GOARCH=amd64 CGO_ENABLED=1 go build -buildmode=plugin -o $(PLUGIN_NA
```

3. 배포 패키징

Docker를 사용한 배포:

```
FROM golang:1.21-alpine AS builder
```

```
# 빌드 의존성 설치
RUN apk add --no-cache gcc musl-dev

WORKDIR /app
COPY . .

# 플러그인 빌드
RUN CGO_ENABLED=1 go build -buildmode=plugin -o my-plugin.so

FROM ipfs/kubo:latest

# 플러그인 복사
COPY --from=builder /app/my-plugin.so /data/ipfs/plugins/

# 권한 설정
RUN chmod +x /data/ipfs/plugins/my-plugin.so

EXPOSE 4001 5001 8080
```

설치 스크립트:

```
#!/bin/bash
```

```
set -e

PLUGIN_NAME="my-custom-plugin"
PLUGIN_VERSION="1.0.0"
IPFS_PLUGINS_DIR="$HOME/.ipfs/plugins"

# 플러그인 디렉토리 생성
mkdir -p "$IPFS_PLUGINS_DIR"

# 아키텍처 감지
ARCH=$(uname -m)
OS=$(uname -s | tr '[:upper:]' '[:lower:]')

case $ARCH in
x86_64) ARCH="amd64" ;;
arm64|aarch64) ARCH="arm64" ;;
*) echo "Unsupported architecture: $ARCH"; exit 1 ;;
esac

# 플러그인 다운로드
PLUGIN_URL="https://github.com/myorg/my-ipfs-plugin/releases/download/v${PLUGIN_

echo "Downloading plugin from $PLUGIN_URL"
```

```
curl -L -o "$IPFS_PLUGINS_DIR/${PLUGIN_NAME}.so" "$PLUGIN_URL"

# 실행 권한 부여
chmod +x "$IPFS_PLUGINS_DIR/${PLUGIN_NAME}.so"

echo "Plugin installed successfully!"
echo "Please restart your IPFS daemon to load the plugin."
```

플러그인 설정 관리

1. 설정 스키마 정의

```
type PluginConfig struct {
Enabled bool `json:"enabled"`
LogLevel string `json:"logLevel"`
Options map[string]interface{} `json:"options"`
Advanced AdvancedConfig `json:"advanced"`
}

type AdvancedConfig struct {
BufferSize int `json:"bufferSize"`
FlushInterval time.Duration `json:"flushInterval"`
MaxRetries int `json:"maxRetries"`
```

```
}


func (p *myCustomPlugin) parseConfig(configData  interface{}) (*PluginConfig, er
if  configData == nil {
return &PluginConfig{
Enabled: false,
LogLevel: "info",
Options: make(map[string]interface{}),
Advanced: AdvancedConfig{
BufferSize: 1024,
FlushInterval: 30 * time.Second,
MaxRetries: 3,
},
}, nil
}
// JSON 언마샬링을 통한 설정 파싱
configBytes, err := json.Marshal(configData)
if  err != nil {
return  nil, fmt.Errorf("failed to marshal config: %w", err)
}
var  config PluginConfig
err = json.Unmarshal(configBytes, &config)
if  err != nil {
```

```
return  nil, fmt.Errorf("failed to unmarshal config: %w", err)
}
return &config, nil
}
```

2. 설정 검증

```
func (p *myCustomPlugin) validateConfig(config *PluginConfig) error {
// 로그 레벨 검증
validLogLevels := []string{"debug", "info", "warn", "error"}
if !contains(validLogLevels, config.LogLevel) {
return  fmt.Errorf("invalid log level: %s", config.LogLevel)
}
// 고급 설정 검증
if  config.Advanced.BufferSize <= 0 {
return  fmt.Errorf("buffer size must be positive")
}
if  config.Advanced.FlushInterval <= 0 {
return  fmt.Errorf("flush interval must be positive")
}
if  config.Advanced.MaxRetries < 0 {
return  fmt.Errorf("max retries cannot be negative")
}
```

```
return nil
}

func contains(slice []string, item string) bool {
for _, s := range slice {
if s == item {
return true
}
}
return false
}
```

3. 설정 예제 파일

config.json:

```
{
"Plugins": {
"Plugins": {
"my-custom-plugin": {
"Disabled": false,
"Config": {
"enabled": true,
"logLevel": "info",
```

```
"options": {
"feature1": true,
"feature2": "custom-value",
"timeout": 30
},
"advanced": {
"bufferSize": 2048,
"flushInterval": "45s",
"maxRetries": 5
}
}
}
}
}
}
```

모범 사례 및 주의사항

1. 보안 고려사항

```
// 입력 검증
func (p *myCustomPlugin) validateInput(input string) error {
if len(input) > maxInputLength {
```

```
return fmt.Errorf("input too long")
}
// 위험한 문자 필터링
if strings.ContainsAny(input, dangerousChars) {
return fmt.Errorf("input contains dangerous characters")
}
return nil
}

// 권한 확인
func (p *myCustomPlugin) checkPermissions(operation string) error {
if !p.hasPermission(operation) {
return fmt.Errorf("insufficient permissions for operation: %s", operation)
}
return nil
}
```

2. 오류 처리

```
// 구조화된 오류 타입
type PluginError struct {
Code string
Message string
```

```
Cause  error
}


func (e *PluginError) Error() string {
if  e.Cause != nil {
return  fmt.Sprintf("%s: %s (caused by: %v)", e.Code, e.Message, e.Cause)
}
return  fmt.Sprintf("%s: %s", e.Code, e.Message)
}


// 오류 래핑
func (p *myCustomPlugin) performOperation() error {
err := someInternalOperation()
if  err != nil {
return &PluginError{
Code: "OPERATION_FAILED",
Message: "Failed to perform internal operation",
Cause: err,
}
}
return  nil
}
```

3. 리소스 관리

```
func (p *myCustomPlugin) Start(api coreiface.CoreAPI) error {
// 컨텍스트 생성
p.ctx, p.cancel = context.WithCancel(context.Background())
// 리소스 초기화
p.resources = make(map[string]io.Closer)
// 정리 함수 등록
runtime.SetFinalizer(p, (*myCustomPlugin).cleanup)
return nil
}

func (p *myCustomPlugin) Close() error {
// 컨텍스트 취소
if p.cancel != nil {
p.cancel()
}
// 리소스 정리
return p.cleanup()
}

func (p *myCustomPlugin) cleanup() error {
var errs []error
```

```
for name, resource := range p.resources {
if err := resource.Close(); err != nil {
errs = append(errs, fmt.Errorf("failed to close %s: %w", name, err))
}
}
if len(errs) > 0 {
return fmt.Errorf("cleanup errors: %v", errs)
}
return nil
}
```

4. 로깅 및 모니터링

```
import (
logging "github.com/ipfs/go-log/v2"
"go.opentelemetry.io/otel"
"go.opentelemetry.io/otel/trace"
)

var log = logging.Logger("my-custom-plugin")
var tracer = otel.Tracer("my-custom-plugin")

func (p *myCustomPlugin) performTrackedOperation(ctx context.Context) error {
```

```
// 분산 트레이싱

ctx, span := tracer.Start(ctx, "performOperation")

defer span.End()

// 구조화된 로깅

log.With("operation", "performTrackedOperation").Info("Starting operation")

err := p.doWork(ctx)

if err != nil {

span.RecordError(err)

log.With("error", err).Error("Operation failed")

return err

}

log.Info("Operation completed successfully")

return nil

}
```

이 가이드를 통해 IPFS Kubo용 커스텀 플러그인을 개발, 테스트, 배포할 수 있습니다. 각 플러그인 타입별 특성을 이해하고 모범 사례를 따르면 안정적이고 효율적인 플러그인을 만들 수 있습니다.